

Defining Class Attributes and Class Methods

Learning Goals

- Use class attributes and methods to write durable and powerful code.
 - Define class attributes and class methods.
 - Understand when, why, and how to use class attributes and class methods.
 - Accomplish complex programming tasks using knowledge from previous modules.
-

Key Vocab

- **Attribute:** variables that belong to an object.
 - **Constant:** variable whose value cannot be changed.
 - **Instance:** one specific working copy of a class. It is created when a class's `__init__` method is called.
 - **Class:** a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
 - **Static:** an attribute or method that cannot manipulate the class or instance it belongs to.
 - **Exception:** an error that occurs during the execution of a program. Exceptions can be anticipated and handled without disrupting the execution of the program.
-

Introduction

All objects are bundles of data and behavior — or attributes and methods. We understand this to be true of instances of a class. Each instance contains attributes or properties to store data, as well as methods that can enact behaviors.

For example, let's say we have a class, `Album`. Every individual album instance should have a release date attribute. To accomplish this, we'll define an instance attribute, `self.release_date` that stores and makes this information available.

```
class Album:

    def __init__(self, date):
        self.release_date = date
```

Here we have an instance attribute, `self.release_date`, which can be accessed through dot notation on the instance itself:

```
album = Album(1991)
album.release_date
# => 1991
```

What you might not know, however, is that the `Album` class itself is also an object. If our definition of an object is a bundle of code that contains attributes and behaviors, then the entire `Album` class itself absolutely qualifies.

The `Album` class can have its own attributes and methods. We call these class attributes and class methods.

Why Use Class Attributes and Methods

Let's say you wanted to keep a counter for how many albums you had in your music collection. That way, you can brag to your friends about what a music aficionado you are. The current code in our `Album` class has no way to keep such a count. Looks like we will have to write some code to accommodate this new feature of our program.

When it comes to adding new features or functionalities to our code, we start out by asking a question: whose responsibility is it to enact this behavior or functionality?

Right now, our program is pretty simple. We have an `Album` class and we have album instances. So, is it the responsibility of an individual album to keep a count of all of the other albums? Or is it the responsibility of the `Album` class, which actually produces the individual albums, to

keep a running count? I think we can agree that it *isn't* the job of the individual albums, but the job of the `Album` class to keep a count of all of the instances it produces.

Now that we've decided whose job it is to enact the "keep a count of all albums" behavior, we can talk about *how* we enact that behavior.

We do so with the use of class attributes and methods. Our goal is to be able to ask the `Album` class: "how many albums have you produced?" When we ask an object to tell us something about itself, we use methods. It would be great if we could do something like:

```
Album.album_count  
# => 0
```

...and return the number of existing albums. Let's build out this capability now.

Building Class Methods and Using Class Attributes

An **instance attribute** is responsible for holding information regarding an *instance*. It is a variable that is available in **scope** for all instance methods in the class.

A **class attribute** is accessible to the entire class — it has **class scope**. A class method is a method that is called on the class itself, not on the instances of that class.

Class attributes are typically used to store information regarding the class as a whole and class methods enact behaviors that belong to the whole class, not just to individual instances of that class.

Defining a Class Attribute

A class attribute is declared using the same notation as anywhere else. We will simply say `album_count = 0`.

What's important and what makes this a class attribute is where it is declared. **A class attribute must be declared outside of any methods in the class.**

Let's create our class attribute now:

```
class Album:

    album_count = 0

    def __init__(self, date):
        self.release_date = date
```

Great, now we have a class attribute to store our count of albums in. Since any `Album` objects will be built from this class, we can access `album_count` through the `Album` class or any `Album` objects that we instantiate using dot notation.

```
joshua_tree = Album(1987)
joshua_tree.album_count
# => 0
Album.album_count
# => 0
```

► If we enter the code `Album.album_count += 1`, what will `Album.album_count` become? How about `joshua_tree.album_count`?

The class attribute exists, but it should be updated whenever we add a new album. Let's build on this class to make it a bit smarter.

Manipulating Class Attributes From Instance Methods

Our `album_count` is stuck at `0`. When and how should we increment it? The count of albums should go up as soon as a new album is created, or initialized. We can hook into this moment in time in our `__init__` method:

```
class Album:

    album_count = 0

    def __init__(self, date):
```

```
Album.album_count += 1
self.release_date = date
```

Here we are using the `album_count` class attribute, inside of our `__init__` method, which is an instance method. We are saying: when a new album is created, access the `album_count` class attribute and increment its value by 1.

Using our class name and dot notation, we can access our class attributes anywhere in our class: in both class and instance methods.

Now our code should behave in the following manner:

```
Album()
Album()
Album()

Album.album_count
# 3
```

We've got an instance method set up now to manipulate our `album_count` class attribute when we instantiate a new album. This is a very useful feature, but what if we already have an album collection and want to manipulate the `album_count` attribute without creating new `Album` objects?

Defining a Class Method

A class method is defined like this:

```
@classmethod
def class_method_name(cls):
    # some code
```

► *What is `@classmethod` telling the interpreter to do?*

Here, the `cls` keyword refers to the **entire class itself**, *not to an instance of the class*. In this case, we are inside the class only, not inside an instance method of that class. So, we are in the **class scope**, not the instance scope.

Let's refactor our `Album` class so that `album_count` can be changed by the class itself:

```
class Album:

    album_count = 0

    def __init__(self, date):
        self.increase_album_count()
        self.release_date = date

    @classmethod
    def increase_album_count(cls, increment=1):
        cls.album_count += increment
```

Now we have an `Album` class that increases the number of albums as we get new ones, but that does so through a method connected to the class itself rather than new objects.

Class Constants

One other type of variable that can be useful when building out classes is a **class constant**. Class constants have a lot in common with class attributes. Both constants and attributes:

- Are defined in the body of the class.
- Can be accessed from within a class method.
- Can be accessed from within an instance method.

A class constant looks a bit different than a class attribute. It's defined using all capital letters, like so:

```
class User:
    ROLES = ["Admin", "Moderator", "Contributor"]
```

When deciding when to use a class constant or a class attribute, the key distinction is that class constants are used to store data that *doesn't change* (is constant), while class attributes are used to store data that does change.

For example, we could define a list of valid genres for our album class using a class constant:

```
class Album:

    GENRES = ["Hip-Hop", "Pop", "Jazz"]
    album_count = 0

    def __init__(self, genre, date):
        if self.check_genre(genre):
            self.increase_album_count()
            self.genre = genre
            self.release_date = date

    @classmethod
    def check_genre(cls, genre):
        return genre in cls.GENRES

    @classmethod
    def increase_album_count(cls, increment=1):
        cls.album_count += increment
```

Scope-wise, class constants can *also* be accessed from outside of the class using this syntax:

```
Album.GENRES
# => ["Hip-Hop", "Pop", "Jazz"]
```

Unlike in JavaScript, declaring a constant variable in Python doesn't actually prevent the variable from being reassigned:

```
Album.GENRES = "not a list anymore"  
Album.GENRES  
# => "not a list anymore"
```

However, declaring a variable with a constant is still a good indicator to other developers that they *shouldn't* reassign the variable's value.

Conclusion

So far in our object-oriented Python code, we've focused on defining behavior that is specific to an individual instance of a class using **instance methods** and **instance attributes**. By also using **class methods**, **class attributes**, and **class constants**, we can expand on our classes' functionality by defining behavior that's not tied to one particular instance of a class, but is related more generally to the class itself.

Resources

- [Python Documentation](https://docs.python.org/3/) ↗ (https://docs.python.org/3/)
- [Classes - Python](https://docs.python.org/3/) ↗ (https://docs.python.org/3/)
- [Python Class Attributes: An Overly Thorough Guide - Toptal](https://www.toptal.com/python/python-class-attributes-an-overly-thorough-guide) ↗ (https://www.toptal.com/python/python-class-attributes-an-overly-thorough-guide)
- [Python's Instance, Class, and Static Methods Demystified - Real Python](https://realpython.com/instance-class-and-static-methods-demystified/) ↗ (https://realpython.com/instance-class-and-static-methods-demystified/)